

Full-Stack Development Training: Checkpoint 2

Time Allowed: 3 Hours | 7 Problems | 100 Marks Total

Instructions

- No internet access. No AI assistance, no code-completion beyond your editor's built-in syntax highlighting, no discussion with other trainees.
- Plain JavaScript only (ES2020+). No frameworks, no npm packages, no live network calls. If a problem needs "the outside world" (an API, a delay), it gives you a mock function to call instead - use exactly that, do not invent your own network calls.
- Create one file per problem, named exactly as given (e.g. p1_rateLimiter.js). Each file must run standalone with node p1_rateLimiter.js.
- Write your solution as the requested named function(s)/class(es), exported with module.exports = { ... } at the bottom of the file. Do not rename the required exports.
- You may write unit tests.
- Partial credit is given for partially correct or partially handled-edge-case solutions. A working solution that misses one edge case scores far better than an empty file.
- Manage your own time. Suggested minutes are a guide, not a hard cutoff - you may move between problems in any order.

Problem 1 - Rate-Limited Memoizer (12 marks, ~20 min)

File: p1_rateLimiter.js

Implement:

JavaScript

```
function createRateLimiter(fn, maxCalls, windowMs) { /* ... */ }
```

createRateLimiter returns a new function with the same signature as fn. Calling the returned function:

- Executes fn and returns its result, **if** fewer than maxCalls calls have happened within the trailing windowMs milliseconds.
- Otherwise throws an Error with the message "Rate limit exceeded" **without** calling fn.
- Correctly "forgets" calls older than windowMs - i.e. the window slides, it does not just reset every windowMs.

You may assume the grading harness controls time using Jest-style fake timers or by calling your function with real, short delays - your implementation must use real wall-clock time (Date.now()), not assume any specific test structure.

Example

JavaScript

```
const limited = createRateLimiter((x) => x * 2, 2, 1000);  
  
limited(1); // 2  
  
limited(2); // 4  
  
limited(3); // throws "Rate limit exceeded"
```

Problem 2 - Custom Error Hierarchy for a Signup Form (13 marks, ~25 min)

File: p2_signupValidation.js

Build a small validation module for a signup form's { username, email, age } input.

- Define three custom error classes, each extends Error: ValidationError (base, has a field property), RequiredFieldError extends ValidationError, and FormatError extends ValidationError.
- Implement validateSignup(input):
 - username: required; if missing, throw RequiredFieldError with field = "username". If present but shorter than 3 characters, throw FormatError with field = "username".
 - email: required (same error type/field rules); if present, must match a basic pattern something@something.something - otherwise FormatError with field = "email".
 - age: required; must be a number ≥ 13 - otherwise FormatError with field = "age" (missing age is still RequiredFieldError).
 - Validation stops and throws on the **first** failing field, checked in the order username → email → age.
 - If everything passes, return { valid: true }.
- Implement safeValidateSignup(input) that calls validateSignup, and instead of throwing, returns { valid: false, field: err.field, message: err.message } on failure, or { valid: true } on success. It must correctly re-throw (not swallow) any error that is **not** a ValidationError.

Example

JavaScript

```
safeValidateSignup({ username: "ab", email: "x@y.com", age: 20 });  
  
// { valid: false, field: "username", message: "..." }  
  
  
safeValidateSignup({ username: "abcd", email: "x@y.com", age: 20  
});  
  
// { valid: true }
```

Problem 3 - Typed EventEmitter (15 marks, ~25 min)

File: p3_eventEmitter.js

Implement a class EventEmitter with:

- on(event, listener) - registers listener for event. Returns this (chainable).
- off(event, listener) - removes that specific listener for that event.
- emit(event, ...args) - calls every listener registered for event, in registration order, with ...args. Returns the number of listeners called.
- once(event, listener) - registers a listener that automatically removes itself after firing exactly once, even if emit is called for that event multiple times synchronously in the same tick.
- Wildcard support: a listener registered via on("*", listener) fires on **every** emit, receiving (event, ...args) - the event name as the first argument.
- Calling off with a listener that was never registered for that event must not throw, and must not affect other listeners.

Example

JavaScript

```
const bus = new EventEmitter();
const log = [];

bus.on("greet", (name) => log.push(`hi ${name}`));
bus.once("greet", (name) => log.push(`once ${name}`));
bus.on("*", (event, name) => log.push(`* ${event} ${name}`));

bus.emit("greet", "Sijin");
bus.emit("greet", "Sijin");

// log: ["hi Sijin", "once Sijin", "* greet Sijin", "hi Sijin", "* greet Sijin"]
```

Problem 4 - Immutable Order Pipeline (15 marks, ~25 min)

File: p4_orderPipeline.js

You are given an array of order objects shaped like:

JSON

```
{ id: 1, customer: "A", items: [{ sku: "X1", qty: 2, price: 50 }],
  status: "pending" }
```

Implement these as **pure functions** - none may mutate their input arguments, and each must be independently testable:

- `orderTotal(order)` → sum of `qty * price` across the order's items.
- `allItemsAcrossOrders(orders)` → a single flat array of every item object across all orders, each item additionally tagged with the parent `orderId` (do not mutate the original item objects - return new objects).
- `markShipped(orders, orderId)` → returns a **new** array where the order matching `orderId` has status: "shipped", all other orders unchanged (same references, to prove no unnecessary copying); if `orderId` isn't found, returns an equivalent new array with no changes.
- `topCustomersBySpend(orders, n)` → returns the top `n` customers as `[{ customer, total }]`, sorted by total spend descending, ties broken by customer name ascending. A customer's total is the sum of `orderTotal` across all their orders.

Write at least one assertion-style check per function under `require.main === module` proving the input array/object was not mutated (e.g. compare a deep copy taken before the call against the original after the call).

Problem 5 - Debounced Search Index (15 marks, ~25 min)

File: `p5_debouncedSearch.js`

Implement, in plain JS with **no DOM** (this will be tested headlessly with fake timers, not in a browser):

- `debounce(fn, delayMs)` - standard debounce: returns a wrapped function that, however many times it's called in quick succession, only actually invokes `fn` once, `delayMs` after the *last* call, with the arguments from that last call.
- `buildSearchIndex(items, fields)` - given an array of objects and a list of field names, returns a function `search(query)` that returns all items where **any** of the specified fields contains query as a case-insensitive substring. Empty or whitespace-only query should return all items.
- `createDebouncedSearcher(items, fields, delayMs, onResults)` - combines the two functions above: returns a function `handleInput(query)`. Each call to `handleInput` resets a `delayMs` timer. When the timer finally fires - i.e., `delayMs` has passed since the *last* call to `handleInput` - run `search()` on whatever query was passed in that last call, and pass the result to `onResults(results)` exactly once. Calls to `handleInput` that happen before the timer fires must never trigger `onResults` on their own; only the final call in a burst produces output.

Example

JavaScript

```
const items = [{ name: "Red Shoes" }, { name: "Blue Hat" }];  
  
const results = [];
```

```
const search = createDebounceSearcher(items, ["name"], 300, (r) =>
  results.push(r));

search("re");

search("red");
```

Problem 6 - Shape Factory & Builder (15 marks, ~25 min)

File: p6_shapeFactory.js

- Build a class hierarchy: Shape (base, has name, abstract-ish area() that throws "Not implemented" if called directly on a plain Shape), and subclasses Circle(radius), Rectangle(width, height), Triangle(base, height) each correctly overriding area().
- Add a **static factory method** Shape.create(type, ...args) that returns the correct subclass instance based on a string type ("circle" | "rectangle" | "triangle"), throwing Error("Unknown shape type: <type>") for anything else.
- Build a ShapeGroupBuilder with a fluent/chainable API: .add(shape), .filterMinArea(min) (drops shapes below a minimum area for the *final* build only - does not mutate previously added shapes), .sortByArea(direction) ("asc"|"desc"), and .build() which returns the final plain array of shapes after filters/sorts are applied, in the order the builder methods were chained.

Example

JavaScript

```
const group = new ShapeGroupBuilder()

  .add(Shape.create("circle", 2))

  .filterMinArea(10)

  .sortByArea("desc")

  .build();
```

Problem 7 - Sequential vs Parallel Async Orchestration (15 marks, ~30 min)

File: p7_asyncOrchestration.js

You are given (already implemented for you, at the top of your file - copy it in) a mock async API:

JavaScript



```
function mockFetchUser(id) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (id <= 0) reject(new Error(`Invalid user id: ${id}`));
      else resolve({ id, name: `User${id}` });
    }, 50);
  });
}

function mockFetchOrders(userId) {
  return new Promise((resolve) => {
    setTimeout(() => resolve([ { orderId: userId * 10 + 1 }, {
      orderId: userId * 10 + 2 } ]), 30);
  });
}
```

Implement:

- `loadUserSequential(id)` - fetches the user, then fetches that user's orders, returning { user, orders }. Must use `async/await`.
- `loadUsersParallel(ids)` - fetches multiple users **concurrently** (not one after another) using `mockFetchUser`, and returns an array of { id, user, error } for **every** id - error set (and user undefined) for ids that failed, user set (and error undefined) for ids that succeeded. The overall function must **not** reject just because one id was invalid; a single failure must not prevent the others from being reported.
- `loadUserWithTimeout(id, timeoutMs)` - races `mockFetchUser(id)` against a timeout; if the user fetch hasn't resolved within `timeoutMs`, rejects with `Error("Timed out")` instead of waiting further (the underlying mock call is allowed to keep running in the background - you are not expected to truly cancel it, just not wait for it).

Your solution will be graded partly on timing behavior - `loadUsersParallel` for 5 ids must complete in roughly the time of *one* `mockFetchUser` call, not five sequential ones.